

Cesium Sandcastle Prompt Cheat Sheet (Modular)

How to use this:

Always include prompt 0; **append the next prompt** to the previous conversation instead of starting over.

Prompt 0 – Global Technical Constraints

Paste this at the **top of every prompt**, every time.

“You are helping me write Cesium Sandcastle code.

Technical requirements:

- Code must be completely in JavaScript (Cesium Sandcastle only accepts JS)
- Do NOT provide HTML files
- Any UI panels or buttons must be created dynamically using JavaScript (e.g. innerHTML)
- Use robust error handling (try/catch, defensive checks for undefined properties)
- Use console.log statements for important actions and state changes
- Code must be copy-paste safe for Cesium Sandcastle
- Organise the code into clearly numbered sections with comments”

Prompt 1 – Create Viewer + Google Photorealistic Tiles + Boundary

Create a Cesium Viewer using Google 3D Photorealistic Tiles.

Requirements:

- Set Cesium.Ion.defaultAccessToken (**insert your token**) at the top of the script
- Log to the console when the token is set
- Do not hard-code secrets outside the script (Sandcastle only) - Treat the token as required infrastructure for all later steps
- Disable the globe
- Disable timeline and animation
- Use the Google geocoder
- Optimise request concurrency for Google tiles

Then add a site boundary as a thick red outline:

- Use the provided latitude/longitude coordinates
- Draw the boundary as a polyline (not a filled polygon)
- The outline should be thick and clearly visible
- No fill, outline only
- Log when the boundary is added

Prompt 2 – Add IFC / 3D Tiles Asset from Cesium ion

Add an IFC / 3D Tiles model from Cesium ion.

Requirements:

- Load the model using its Cesium ion asset ID (**Insert your asset ID**)
- Add it to the scene as a primitive
- Wait for the tileset to be ready before using it
- Store a reference to the tileset for later interaction
- Zoom the camera to the model once loaded
- Log successful load and failures

Prompt 3 – Add Camera View Buttons (Top / Front / Back / Side)

Add a views widget to the bottom-left of the screen.

Requirements:

- Buttons: Top, Front, Back, Side
- Buttons must be created dynamically with JavaScript
- Use a helper function to fly the camera relative to the IFC model
- Camera movement should use flyToBoundingSphere
- Views should be based on heading, pitch, and range
- Include smooth animation
- Log which view is activated

Do not hard-code world coordinates.

Do not break existing widgets.

Prompt 4 – Toggle IFC Model Visibility

Add a toggle control to show/hide the IFC model.

Requirements:

- Add the toggle to the existing Layers panel
- Checkbox should toggle the IFC model visibility on and off
- Use the stored tileset reference
- Ensure toggling works even if the model loads late
- Log visibility changes

Do not remove any existing UI.

Prompt 5 – Feature-Level Clicking (IFC Parts)

Modify click interaction so that only the clicked IFC element is highlighted.

Requirements:

- Clicking an IFC part should highlight ONLY that part in yellow
- Previously highlighted elements should return to their original color
- Do not recolor the entire tileset
- Use Cesium3DTileFeature for feature-level interaction
- Log IFC properties of the clicked feature (if available)
- Keep existing InfoBox behaviour unchanged

Use defensive checks to avoid runtime errors.

Prompt 6 – Add Robust Logging & Error Guards

Review the existing code and improve robustness.

Requirements:

- Add try/catch blocks where async operations occur
- Add defensive checks before accessing properties

- Add clear console.log messages for:
 - Viewer creation
 - Tileset loading
 - UI creation
 - Camera actions
 - User interactions

Do not add new features.

Only improve reliability and clarity.

Prompt 7 - Prepare the Cesium Sandcastle app for external data integration.

Requirements:

- Do NOT change any visual behaviour
- Add clearly named variables and placeholders for external data
- Ensure global variables are defined in a safe scope
- Add console.log statements indicating readiness for external data

Do not fetch or load any data yet.

This is preparation only.

Prompt 8 – Load CSV data from GitHub

Requirements:

- Use fetch() to load a CSV from a raw GitHub URL (I will provide the URL)
- Parse the CSV using JavaScript only (no external libraries)
- Use a lookup object keyed by globalId
- Ensure the CSV is loaded only once
- Add robust error handling for:
 - Network errors
 - Empty files
 - Unexpected formats

- Log:
 - CSV URL
 - Detected headers
 - Number of records loaded

Add this as a clearly numbered new section.

Do not connect the CSV to the IFC yet.

Prompt 9 – Inspect IFC Metadata for Linking

Requirements:

- When an IFC element is clicked, safely inspect its available properties
- Log property names and example values to the console
- Use defensive checks (some features may not expose properties)
- Do NOT change highlighting or InfoBox behaviour

This prompt is for inspection and learning only.

Prompt 10 – Link IFC Elements to CSV Using globalId

Goal:

Create the digital-twin “join”.

Connect IFC elements to external CSV data using globalId.

Requirements:

- Use the IFC element’s globalId as the join key
- Match it against rows in the previously loaded CSV
- Handle both "globalId" and "GlobalId" casing safely
- Log whether a match is found or not
- Do NOT modify any geometry or styling yet

Add this logic inside the existing click interaction.

Prompt 11 – Display CSV Data in the InfoBox on Click

Goal:

Surface external data to the user.

Extend the IFC click interaction so that CSV data is shown in the InfoBox.

Requirements:

- When an IFC element with matching CSV data is clicked:
 - Display the CSV data in a table format in the InfoBox
- Reuse a single InfoBox entity (do not create new ones repeatedly)
- Do not manipulate the DOM directly
- Ensure InfoBox updates cleanly across multiple clicks
- Log when CSV data is successfully displayed

Keep all existing highlight and interaction behaviour unchanged.

*****Prompt 12 – Modular Scaling Prompt (Reusable)**

This is the **generic scaling prompt** you can reuse.

Extend the existing Cesium Sandcastle app with a new feature.

Rules:

- Do not rewrite or remove existing sections
- Add the new feature as a clearly numbered section
- Reuse existing helpers and state where possible
- Follow the same UI and logging patterns already in use
- Keep the code modular and readable

The new feature is:

(describe your own new feature here)